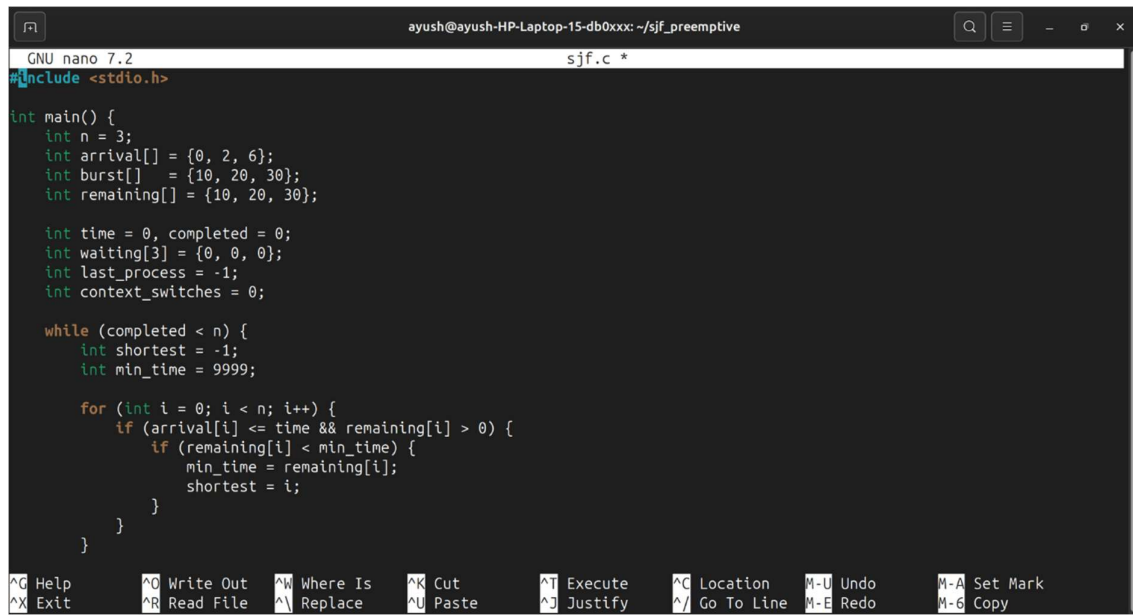


Practical-5 OUTPUT

Initialization:

This section initializes the process count along with arrival time, burst time, and remaining execution time for SJF pre-emptive scheduling.



```
GNU nano 7.2 sjf.c *
#include <stdio.h>

int main() {
    int n = 3;
    int arrival[] = {0, 2, 6};
    int burst[] = {10, 20, 30};
    int remaining[] = {10, 20, 30};

    int time = 0, completed = 0;
    int waiting[3] = {0, 0, 0};
    int last_process = -1;
    int context_switches = 0;

    while (completed < n) {
        int shortest = -1;
        int min_time = 9999;

        for (int i = 0; i < n; i++) {
            if (arrival[i] <= time && remaining[i] > 0) {
                if (remaining[i] < min_time) {
                    min_time = remaining[i];
                    shortest = i;
                }
            }
        }

        if (shortest == -1) {
            time++;
            continue;
        }

        if (last_process != shortest && last_process != -1) {
            context_switches++;
        }

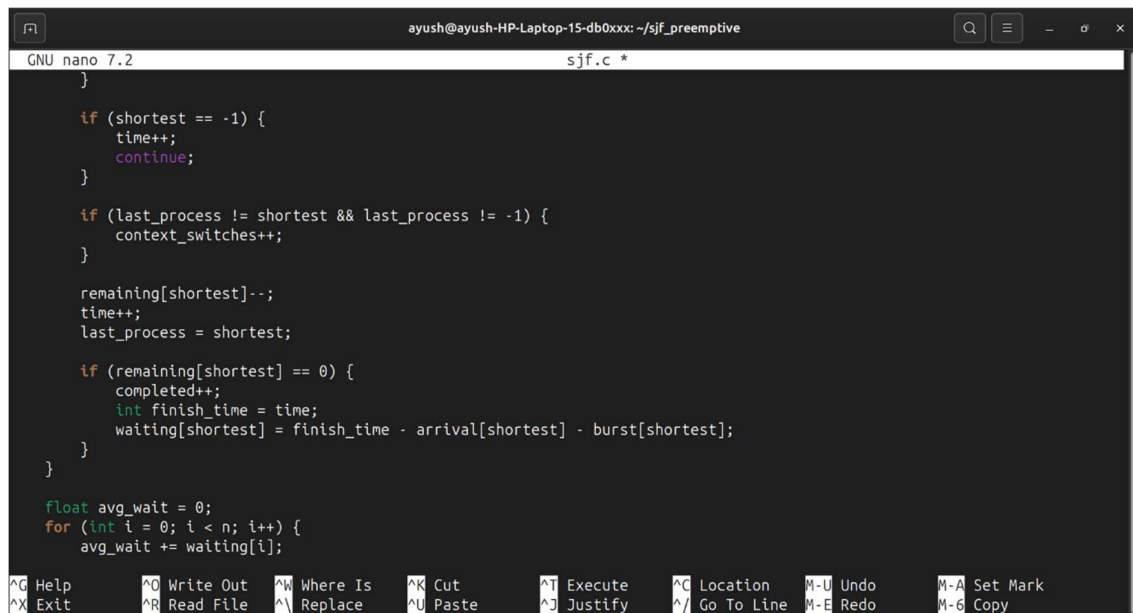
        remaining[shortest]--;
        time++;
        last_process = shortest;

        if (remaining[shortest] == 0) {
            completed++;
            int finish_time = time;
            waiting[shortest] = finish_time - arrival[shortest] - burst[shortest];
        }
    }

    float avg_wait = 0;
    for (int i = 0; i < n; i++) {
        avg_wait += waiting[i];
    }
}
```

Scheduling:

This section selects the process with the shortest remaining execution time and handles preemption and context switching during execution.



```
GNU nano 7.2 sjf.c *
    }

    if (shortest == -1) {
        time++;
        continue;
    }

    if (last_process != shortest && last_process != -1) {
        context_switches++;
    }

    remaining[shortest]--;
    time++;
    last_process = shortest;

    if (remaining[shortest] == 0) {
        completed++;
        int finish_time = time;
        waiting[shortest] = finish_time - arrival[shortest] - burst[shortest];
    }
}

float avg_wait = 0;
for (int i = 0; i < n; i++) {
    avg_wait += waiting[i];
}

printf("Average waiting time: %.2f\n", avg_wait);
return 0;
}
```

Calculation:

This section computes the waiting time of each process and calculates the average waiting time after all processes complete execution.



```
GNU nano 7.2 sjf.c *
}

remaining[shortest]--;
time++;
last_process = shortest;

if (remaining[shortest] == 0) {
    completed++;
    int finish_time = time;
    waiting[shortest] = finish_time - arrival[shortest] - burst[shortest];
}

}

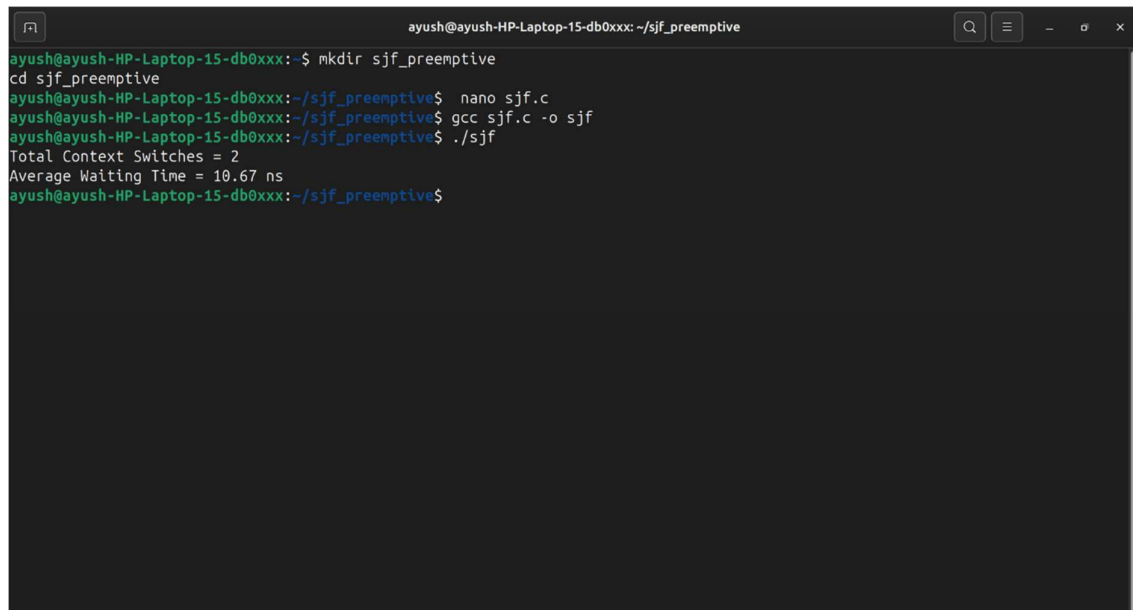
float avg_wait = 0;
for (int i = 0; i < n; i++) {
    avg_wait += waiting[i];
}
avg_wait /= n;

printf("Total Context Switches = %d\n", context_switches);
printf("Average Waiting Time = %.2f ns\n", avg_wait);

return 0;
}
```

Output:

This section displays the final result of the SJF (Pre-emptive) scheduling program, including the total number of context switches and the average waiting time.



```
ayush@ayush-HP-Laptop-15-db0xxx: ~/sjf_preemptive
ayush@ayush-HP-Laptop-15-db0xxx:~$ mkdir sjf_preemptive
ayush@ayush-HP-Laptop-15-db0xxx:~/sjf_preemptive$ nano sjf.c
ayush@ayush-HP-Laptop-15-db0xxx:~/sjf_preemptive$ gcc sjf.c -o sjf
ayush@ayush-HP-Laptop-15-db0xxx:~/sjf_preemptive$ ./sjf
Total Context Switches = 2
Average Waiting Time = 10.67 ns
ayush@ayush-HP-Laptop-15-db0xxx:~/sjf_preemptive$
```